Agendamento de monitorias

Engenharia de Software - A2
Thiago Wesley Pasquim de Melo e André Nacamura

1. Projeto inicial

Projeto inicial — 18/10

1. História escolhida e escopo

História 1 (Prioridade Alta)**:** Como aluno, eu quero escolher uma disciplina, ver os horários livres e reservar um atendimento para ser atendido pelo monitor sem conflito de horário.

A história foi escolhida porque a reserva é a necessidade central do cliente, que afirma: “Se a reserva de horários funcionar, já ajuda muito.” O protótipo ficará restrito à consulta e à reserva, com os cinco critérios de aceitação registrados na etapa anterior.

A solução será desenvolvida em Python, com dados em memória, sem necessidade de interface gráfica ou entrada pelo teclado. Alunos, disciplinas e horários serão preparados previamente, permitindo que os futuros testes chamem diretamente a lógica. Cada atendimento terá duração de 30 minutos e capacidade para uma ou mais pessoas. Cadastro, cancelamento pelo monitor, notificações e relatórios não serão implementados nesta história; horários previamente marcados como cancelados serão reconhecidos nas consultas e reservas.

2. Classes e responsabilidades

Aluno: identifica quem solicita a reserva, por RA e nome.

Disciplina: identifica a matéria escolhida e permite agrupar seus horários.

Horário de atendimento: reúne disciplina, monitor responsável, data, início, capacidade e indicação de cancelamento. Calcula as vagas disponíveis pela capacidade menos as reservas existentes; se estiver cancelado, fica indisponível.

Reserva: associa um aluno a um horário, ocupando uma vaga. A disciplina e a data são obtidas do horário associado.

Serviço de agendamento: mantém os dados e oferece funções para listar disciplinas, consultar horários livres, contar reservas semanais e efetuar uma reserva. Antes de registrar, verifica a existência do aluno e do horário, o cancelamento, as vagas e o limite semanal. Uma recusa gera erro descritivo sem modificar os registros.

3. Fluxo e decisões iniciais

O aluno escolhe a disciplina, consulta os horários e solicita a reserva. O serviço valida as condições e registra a reserva, ocupando uma vaga. Em caso de erro, os dados não mudam. Como o cliente não definiu o limite semanal, adotaremos uma reserva por aluno e disciplina, com valor configurável. A semana vai de segunda a domingo, pela data do atendimento. Reservas em horários cancelados não contam para o limite.

4. Previsão dos testes por critério de aceitação

Os testes de aceitação serão implementados posteriormente com unittest. A previsão abaixo faz parte do projeto inicial exigido no passo 5 do enunciado.

1. **Listar apenas horários da disciplina com vagas:** o teste de consulta preparará horários de duas disciplinas, incluindo um lotado e um cancelado. Espera-se somente o horário disponível da disciplina escolhida.
2. **Registrar a reserva e diminuir uma vaga:** o teste de reserva válida usará um horário com duas vagas. Espera-se uma reserva vinculada ao aluno e uma vaga restante.
3. **Recusar horário sem vagas:** o teste de lotação tentará reservar um horário cheio. Espera-se erro, sem criar ou alterar reservas.
4. **Recusar horário cancelado:** o teste de cancelamento tentará reservar um horário cancelado com capacidade restante. Espera-se erro e nenhuma nova reserva.
5. **Respeitar o limite semanal configurável:** os testes do limite verificarão a recusa da segunda reserva na mesma disciplina e semana, a permissão em outra semana ou disciplina e a permissão da segunda reserva quando o limite for alterado para dois.

O detalhamento dos testes fica para 19/10, conforme o cronograma.

Projeto inicial - 19/10

1. Continuação do projeto inicial

Este documento complementa o projeto de 18/10 com o mapeamento dos testes da história escolhida, mantendo o escopo, as classes e as responsabilidades já descritos.

História 1 — Prioridade Alta: Como aluno, eu quero escolher uma disciplina, ver os horários livres e reservar um atendimento para ser atendido pelo monitor sem conflito de horário.

Os testes serão escritos posteriormente com unittest e chamarão diretamente as funções do serviço de agendamento. Cada teste terá seus próprios dados em memória, sem depender da execução dos demais ou de entrada pelo teclado.

2. Mapeamento dos critérios de aceitação

Os nomes abaixo são os nomes previstos para os futuros testes. Os cenários descrevem a preparação, a ação e o resultado esperado; ainda não representam testes implementados ou executados.

CA1 — Listar apenas horários da disciplina escolhida com vagas

Teste: `test_consulta_filtra_disciplina_vagas_e_cancelamento`

Preparar horários de Cálculo e Física, incluindo um de Cálculo disponível, um lotado e um cancelado com vagas. Consultar Cálculo e verificar que somente seu horário disponível é retornado.

Tipo: sucesso.

Teste: `test_consulta_sem_horarios_disponiveis`

Consultar uma disciplina cujos horários estão lotados ou cancelados. Esperar uma lista vazia, sem erro.

Tipo: sucesso, sem disponibilidade.

CA2 — Registrar a reserva para o aluno e diminuir uma vaga

Teste: `test_reserva_registra_aluno_e_reduz_vagas`

Reservar um horário com duas vagas para um aluno sem reservas anteriores. Verificar uma nova reserva vinculada ao aluno e ao horário, com apenas uma vaga restante.

Tipo: sucesso.

CA3 — Recusar horário sem vagas, preservando as reservas existentes

Teste: `test_reserva_recusa_horario_lotado`

Tentar reservar um horário já preenchido por outro aluno. Esperar um erro descritivo, mantendo as reservas existentes e o número de vagas.

Tipo: falha ou exceção.

CA4 — Recusar reserva em horário cancelado

Teste: `test_reserva_recusa_horario_cancelado`

Tentar reservar um horário cancelado que ainda teria vagas pela capacidade. Esperar um erro descritivo, sem criar uma reserva ou alterar os registros existentes.

Tipo: falha ou exceção.

CA5 — Respeitar o limite semanal configurável por aluno e disciplina

Teste: `test_limite_recusa_segunda_reserva_na_mesma_semana`

Com limite de uma reserva, preparar uma reserva na segunda-feira, 19/10/2026. Tentar reservar outro horário da mesma disciplina no domingo, 25/10/2026, para o mesmo aluno. Esperar erro e nenhuma alteração nos registros.

Tipo: falha ou exceção.

Teste: `test_limite_permite_reserva_em_outra_semana`

Preparar uma reserva no domingo, 18/10/2026, e solicitar outra na segunda-feira, 19/10/2026, para o mesmo aluno e disciplina. Esperar sucesso, pois são semanas diferentes.

Tipo: sucesso.

Teste: `test_limite_permite_reserva_em_outra_disciplina`

Preparar uma reserva em Cálculo e solicitar uma em Física para o mesmo aluno, na mesma semana e em outro horário. Esperar sucesso, pois o limite é por disciplina.

Tipo: sucesso.

Teste: `test_limite_configuravel_permite_duas_reservas`

Configurar o limite como dois e preparar uma reserva. Solicitar a segunda na mesma disciplina e semana, esperando sucesso; tentar a terceira e esperar erro, preservando as duas anteriores.

Tipo: sucesso e falha ou exceção.

Teste: `test_limite_desconsidera_reserva_em_horario_cancelado`

Preparar uma reserva associada a um horário cancelado. Com limite de uma reserva, solicitar outro horário disponível da mesma disciplina e semana. Esperar sucesso, pois a reserva do horário cancelado não conta para o limite.

Tipo: sucesso.

3. Decisões mantidas e conclusão da etapa

O limite será de uma reserva por aluno e disciplina por semana, de segunda a domingo, com valor configurável. As vagas correspondem à capacidade menos as reservas. Horários cancelados ficam indisponíveis, e recusas não alteram os registros.
Os cinco critérios têm testes previstos de sucesso ou falha. Cadastro, cancelamento pelo monitor, notificações e relatórios ficam fora desta história, com os dados preparados previamente.

2. Relatório de execução

Integrantes: Thiago Wesley Pasquim de Melo e Andre Nacamura.

Implementamos a História 1: consultar horários e reservar monitorias, verificando vagas, cancelamento, duplicidade e limite semanal configurável por aluno e disciplina.

Como executar

Usamos Python 3.15, com sqlite3 e unittest, sem bibliotecas extras. Na pasta do repositório, execute:

python codigo/principal.py

O programa usa dados de exemplo, sem entrada pelo teclado. Cria o banco monitorias.db e mantém as reservas entre execuções, recusando duplicadas.

Para rodar os testes:

python -m unittest discover -s codigo -p "test_*.py" -v

Em 22/10, os 14 testes passaram, usando bancos temporários separados e apagados ao terminar.

Data	O que foi feito	Responsável
17/10	Cinco histórias no GitHub	Andre
18/10	Escolha da história e projeto das classes.	Thiago
19/10	Thiago: planejamento dos testes, classes e criação do banco. Andre: integração, regras e demonstração.	Ambos
20/10	Thiago: seis testes. Andre: oito testes e bloqueio de reserva duplicada.	Ambos
21/10	Thiago: README e organização das histórias. Andre: documentação dos testes.	Ambos
22/10	Revisão, PDF e conferência do ZIP extraído em outra pasta. Envio no Moodle pendente.	Ambos

Os commits até 21/10 seguiram as datas previstas, sem atraso registrado. Dividimos as tarefas dentro das etapas, embora o plano indicasse ambos em todas elas.

Mudanças no projeto

Adotamos SQLite para manter as reservas e repositorio.py para separar o SQL das regras. Os testes usam arquivos temporários porque a inicialização fecha a conexão. Acrescentamos o bloqueio de reservas duplicadas e quatro testes aos dez previstos. O limite semanal segue em um, configurável. As demais histórias ficaram fora do protótipo.

3. Documentação técnica

1. Histórias de usuário

Definimos cinco histórias e escolhemos a História 1 para implementar.

1.1. Reserva de atendimento

Como aluno, eu quero escolher uma disciplina, ver os horários livres e reservar um atendimento para ser atendido pelo monitor sem conflito de horário.

Critérios de aceitação:

- CA1: Mostrar somente horários da disciplina escolhida com vagas.
- CA2: Registrar a reserva para o aluno e diminuir uma vaga.
- CA3: Recusar horário lotado sem alterar as reservas existentes.
- CA4: Recusar reserva em horário cancelado.
- CA5: Permitir uma reserva por aluno na mesma disciplina por semana, com limite configurável.

Prioridade: Alta. É a função central do sistema e ajuda a organizar a distribuição dos alunos.

Trecho do e-mail:

> A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento.

1.2. Cancelamento pelo monitor

Como monitor, eu quero cancelar um horário em que não posso atender para que os alunos reservados sejam avisados antes de chegarem à sala.

Critérios de aceitação:

- O monitor só pode cancelar os próprios horários.
- O horário fica marcado como cancelado e sai da lista de horários livres.
- Os alunos reservados recebem um aviso.
- Cancelar novamente o mesmo horário gera erro.

Prioridade: Alta. Evita que os alunos cheguem à sala sem saber do cancelamento.

Trecho do e-mail:

> Os alunos reclamam que só descobrem o cancelamento quando chegam na sala.

1.3. Cadastro de horários

Como monitor, eu quero cadastrar os horários em que posso atender, definindo o número de vagas, para que os alunos possam reservá-los em atendimento individual ou em grupo.

Critérios de aceitação:

- Informar disciplina, dia e início, com duração fixa de 30 minutos.
- Recusar horários que se sobreponham para o mesmo monitor.
- Exigir pelo menos uma vaga; valores menores geram erro.
- Mostrar o horário cadastrado na consulta de horários livres.

Prioridade: Média. Os horários são necessários, mas podem ser preparados no código na primeira versão.

Trecho do e-mail:

> No início do semestre, cada monitor informa os horários em que pode atender.

1.4. Relatório de horas

Como coordenador, eu quero consultar o histórico e o total mensal de horas atendidas por cada monitor para contabilizar e pagar as bolsas.

Critérios de aceitação:

- Somar as horas por monitor no mês escolhido.
- Desconsiderar atendimentos cancelados.
- Permitir acesso somente ao coordenador; outros usuários recebem erro de permissão.
- Mostrar zero horas quando não houver atendimentos.

Prioridade: Média. É necessário para as bolsas, mas depende dos atendimentos registrados.

Trecho do e-mail:

> A coordenação precisa de um relatório mensal com as horas atendidas por cada monitor, porque isso conta para o pagamento da bolsa.

1.5. Consulta dos conteúdos com mais dúvidas

Como professor, eu quero ver quais conteúdos geraram mais atendimentos na minha disciplina para reforçá-los em sala de aula.

Critérios de aceitação:

- O monitor registra o conteúdo após o atendimento.
- Mostrar os conteúdos do mais atendido para o menos atendido.
- Permitir a consulta somente dos conteúdos da disciplina do professor.
- Agrupar atendimentos sem conteúdo como “Não informado”.

Prioridade: Baixa. Ajuda no planejamento das aulas, mas não é necessária para as reservas.

Trecho do e-mail:

> Os professores das disciplinas gostariam de saber quais conteúdos geram mais dúvidas, para reforçar em aula.

2. Testes de aceitação

Os testes estão em codigo/test_reservas.py e usam unittest. Chamam diretamente o serviço e o repositório, com um banco temporário para cada teste.

2.1. Testes por critério

CA1 — Consulta de horários

- **test_consulta_filtra_disciplina_vagas_e_cancelamento:** retorna apenas o horário disponível da disciplina escolhida. Sucesso**.**
- **test_consulta_sem_horarios_disponiveis:** retorna uma lista vazia quando não há vagas. Sucesso**.**

CA2 — Registro da reserva

- **test_reserva_registra_aluno_e_reduz_vagas:** grava a reserva do aluno e reduz uma vaga. Sucesso**.**

CA3 — Horário lotado

- **test_reserva_recusa_horario_lotado:** gera HorarioLotadoError e preserva as reservas existentes. Falha ou exceção.

CA4 — Horário cancelado

- **test_reserva_recusa_horario_cancelado:** gera HorarioCanceladoError sem gravar uma reserva. Falha ou exceção.

CA5 — Limite semanal

- **test_limite_recusa_segunda_reserva_na_mesma_semana:** recusa a segunda reserva na mesma disciplina e semana, mantendo os registros**. Falha ou exceção.**
- **test_limite_configuravel_permite_duas_reservas:** com limite dois, aceita duas reservas e recusa a terceira. Sucesso e falha.
- **test_limite_permite_reserva_em_outra_semana:** permite reservas no domingo e na segunda-feira seguinte. Sucesso.
- **test_limite_permite_reserva_em_outra_disciplina:** permite reservar outra disciplina na mesma semana. Sucesso.
- **test_limite_desconsidera_reserva_em_horario_cancelado:** não conta a reserva de um horário cancelado para o limite. Sucesso.

2.2. Testes adicionais

- **test_reserva_persiste_apos_reabrir_banco:** verifica se a reserva permanece após fechar e reabrir o banco. Sucesso.
- **test_reserva_recusa_mesmo_aluno_no_mesmo_horario:** recusa reserva duplicada sem alterar os registros. Falha ou exceção.
- **test_reserva_permite_outro_aluno_no_mesmo_horario_em_grupo:** permite dois alunos em um horário com duas vagas. Sucesso.

- **test_reserva_recusa_horario_inexistente:** gera HorarioNaoEncontradoError sem alterar os registros. Falha ou exceção.

Os 14 testes passaram. Nos cenários de falha, o teste verifica se o sistema gera o erro esperado e preserva as reservas.

2.3. O que não foi coberto

- **Histórias 2 a 5:** não foram implementadas, pois escolhemos somente a História 1.
- **Saída do programa principal:** foi conferida manualmente em duas execuções, sem teste automático.
- **Operações isoladas do banco:** foram verificadas junto com o serviço, sem testes separados para cada comando SQL.
- **Segunda e domingo da mesma semana:** testamos a mudança entre semanas, mas não essa combinação dentro da mesma semana.
- **Reservas simultâneas:** ficaram fora dos testes porque o protótipo não oferece acesso simultâneo.
- **Duração e capacidade inválida:** não receberam testes específicos, pois o foco foi a reserva, não o cadastro.
- **Duplicidade por gravação direta no banco:** a regra está no serviço e não é aplicada quando o repositório é usado diretamente.